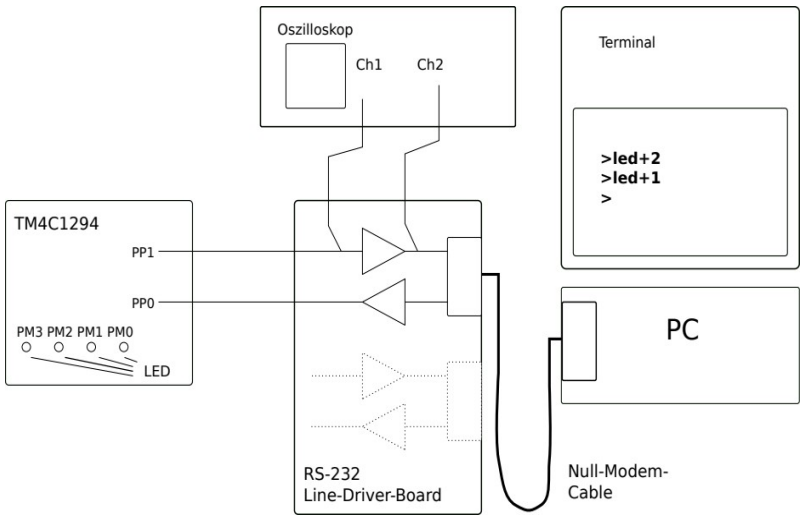
Lab Session 4

UART Protocol

Table of Contents

1 Objectives of the Laboratory Experiment

First, the connection from the MC to the PC is established via the UART. A terminal program is used on the PC. The text entries in the terminal program are transmitted to the microcontroller. Then, several control commands are sent from the terminal program to the microcontroller. These are evaluated there in order to switch individual LEDs on or off.



MC Lab 4 - Hardware Setup

2 Preparation

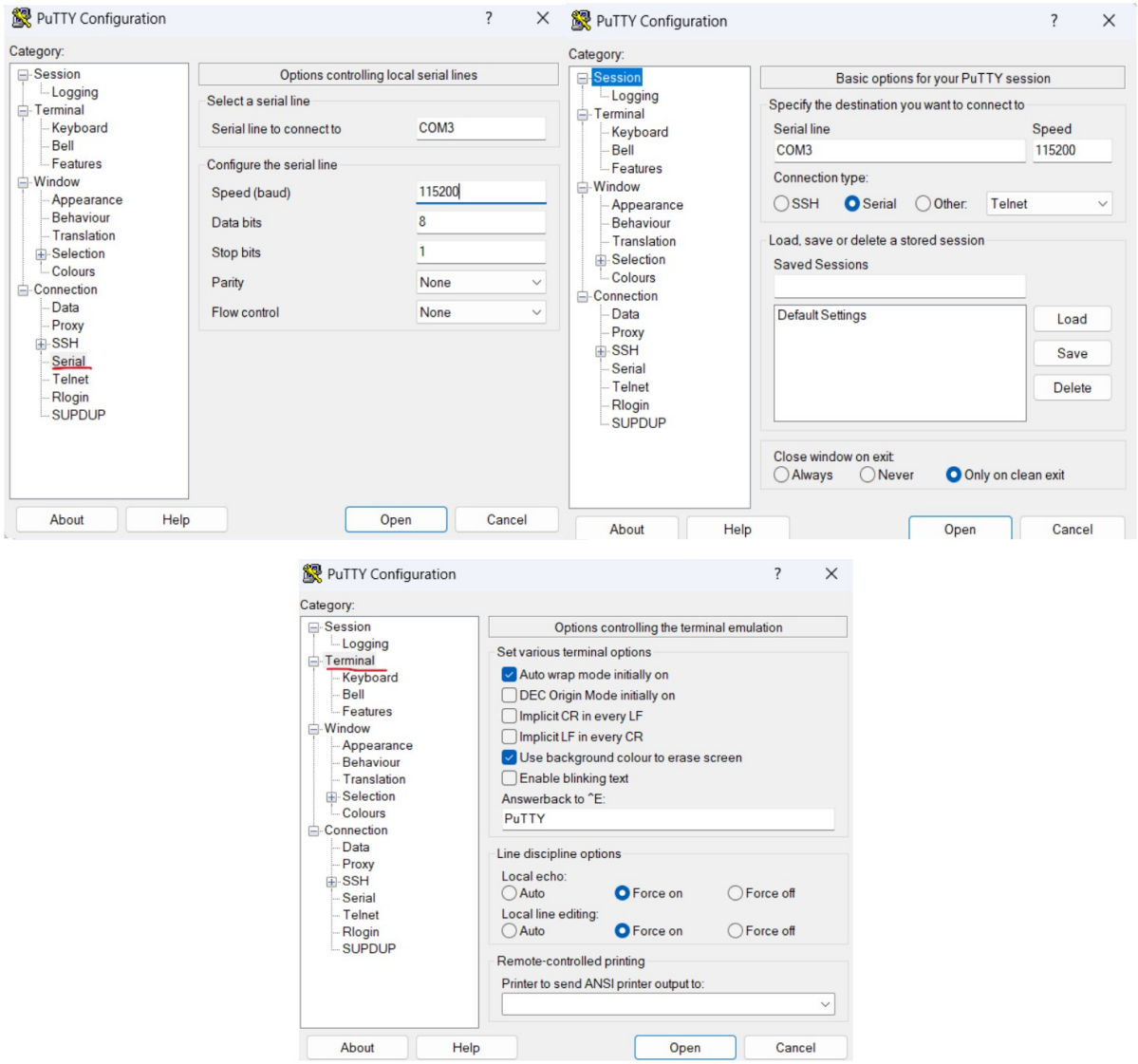
Read through the register configuration of the UART and the examples in the lecture notes. Find the registers listed below in the data sheet and make a note of the relevant bits and the required entries and information:
In addition, Table 10-2 on page 744 and the register description on page 788 are required.

Register	Function	Page in Datasheet
SYSCTL_RCGCUART_R / RCGCUART	UART Run Mode Clock Gating Control	388
UARTx_CTL_R / UARTCTL	UART Control	1188
UARTx_IBRD_R / UARTIBRD	UART Integer Baud-Rate Divisor	1184
UARTx_FBRD_R / UARTFBRD	UART Fractional Baud-Rate Divisor	1185
UARTx_LCRH_R / UARTLCRH	UART Line Control	1186
UARTx_FR_R / UARTFR	UART Flag	1180
UARTx_DR_R / UARTDR	UART Data	1175
GPIO_PORTx_AHB_AFSEL_R / GPIOAFSEL	GPIO Alternate Function Select	770
GPIO_PORTx_AHB_PCTL_R	GPIO Port Control	770

3 Lab Task 1: Establishing and testing the UART Protocol

3.1 Configuring the terminal program

A terminal program must be used so that the PC can access the serial interfaces. The program “Putty” is used in the lab, which is available for most operating systems. The following Figure shows the initial settings required. You can save and restore settings under a selectable name.



3.2 Establish the connection between PC and MC

The next step is to test the connection between the microcontroller and the PC. To do this, a microcontroller program should continuously send a single character. Copy an existing project (or the default project) in the Code Composer Studio development environment and rename the copy of the project with your own project name. Insert the program code from Listing 1 there. The program performs the following actions:

- Configure the parameters of the UART protocol and output UART6 Tx on port P1
- The RC oscillator on the chip with 16 MHz is used.
- Continuously send the character ‘H’ to the PC

Compile the program and load it onto the microcontroller. Test whether the character “H” appears continuously in the terminal program.

```
//=============================================================
// Test program UART6 TX 8/N/1 @ 115200 bit via Port PP1
// LTL 17.5.2020 / mod. V0.2 - V0.4 RMS  1.6.2023
//=============================================================
#include "inc/tm4c1294ncpdt.h"          // Header of the controller type
#include <stdint.h>                     // Header w. types for the register ..

#define IDLETIME 1000                   // waiting time between transmissions
int wt = 0;                             // auxillary variable

void config_port(void){
    // initialize Port P
       SYSCTL_RCGCGPIO_R |= 0x02000;    // switch on clock for Port P
       wt++;                            // delay for stable clock
       // initialize Port P
      GPIO_PORTP_DEN_R |= 0x2;          // enable digital pin function for PP1
      GPIO_PORTP_DIR_R |= 0x2;          // set PP1 to output
      GPIO_PORTP_AFSEL_R |= 0x2;        // switch to alternate pin function PP1
      GPIO_PORTP_PCTL_R |= 0x10;        // select alternate pin function PP1->U6Tx
}

void config_uart(void){
    // initialize UART6
    SYSCTL_RCGCUART_R |= 0x40;          // switch on clock for UART6
    wt++;                               // delay for stable clock
    UART6_CTL_R &= ~0x01;               // disable UART6 for config
    // initialize bitrate of 115200 bit per second
    UART6_IBRD_R = 8;                   // set DIVINT of BRD floor(16 MHz/16*115200 bps)
    UART6_FBRD_R = 44;                  // set DIVFRAC of BRD remaining fraction divider
    UART6_LCRH_R = 0x00000060;          // serial format 8N1
    UART6_CTL_R |= 0x0101;              // UART transmit on and UART enable
}

void idle()      {                      // simple wait for idle state
   int i;
   for (i=IDLETIME;i>0;i--);            // count down loop for waiting
}

void main(void){
    config_port();  // configuration of  Port P
    config_uart();  // configuration of  UART6
    while(1){
        while((UART6_FR_R & 0x20) !=0); // till transmit FIFO not full
        UART6_DR_R = 'H';               // send the character 'H'
        idle();                         // idle time
    }
}
```

3.3 Baud rates and data format

Transmit the following Data in the specified baud rate and format from the MC to the PC. You have to modify the *config_uart()* function, as well as the terminal setup on the PC.

a) ASCII character X with 9600 bps, format 7E1

b) ASCII character a with 38400 bps, format 8O1

c) The binary data 0x3B with 4800 bps, format 7N2

4 Lab Task 2: Receiving a character string from the PC

Modify the program from task 1. Activate the transmitter (Tx) and receiver (Rx) of the UART module and connect both to port P and pins PP1 and PP0, respectively.

Requirement 1: The three characters should be sent with each new line in the terminal as a prompt:

1. Character 0x0D = '\r' (carriage return)
2. Character 0x0A = '\n' (line feed)
3. Character '>' (a symbol as a prompt)

Requirement 2: UART6 Rx must be queried continuously by your program.

Requirement 3: Characters received via UART6 Rx must be stored in an array of type char.

Requirement 4: MAXSIZE must be defined as the size of the array using a macro (10-50).

Requirement 5: When the MAXSIZE-1 character or the value 0x0D is received, the last value of the string/array must be written with 0x00 (\0).

Requirement 6: Then output the string to the console using the printf() function.

Requirement 7: The process should start with a new prompt after output.

Help with errors

The prompt '>' does not appear in the terminal window?

Then the UART module may be incorrectly configured on the transmit side.
Is the Tx output activated and connected to port pin PP1?
Are you stuck in the infinite loop of the interrupt handler FaultISR()? Then you may not have activated the clock of one of the peripheral modules.

Printf() in the CCS console window does not produce any output?

Set a breakpoint where the array is filled from the UART FIFO. Each time a key is pressed in the terminal window, your program should stop there and the character should appear in the array. Check this with the debugger.

The array is not being filled?

Then the UART module may be incorrectly configured on the receiving end.
Is the Rx input activated and connected to port pin PP0 with a cable?
Are you reading the UART6_FR_R register correctly? Does your program hang when querying the Rx FIFO? Check this with a step-by-step run in the debugger.

The array is filled, but not output to the console?

Are you passing the string correctly to the printf() function? Is the string terminated with the character \0?

5 Lab Task 3: Decoding a control command from the PC

The program from Task 2 is to be expanded to interpret the received character string as control commands on the microcontroller.

Requirement 1: The valid input format has the following syntax structure:

led <+|-><0|1|2|3>

The commands are to be interpreted according to the following table.

Command	Function
led+0	Turn on 1st LED on Port M
led+1	Turn on 2nd LED on Port M
led+2	Turn off 3rd LED on Port M
led+3	Turn on 4th LED on Port M
led-0	Turn off 1st LED on Port M
led-1	Turn on 2nd LED on Port M
led-2	Turn off 3rd LED on Port M
led-3	Turn on 4th LED on Port M

Requirement 2: The LEDs on the board are switched on or off individually by the program on the microcontroller after decoding the received character string.

Requirement 3: If the command syntax is violated, command decoding should be aborted and the input ignored.

Requirement 4: The process repeats and starts again with a new prompt '>' on a new line at the terminal.

Requirement 5: The received character string must also be output with printf().

Appendix: ASCII-Tabelle

The following table shows the characters and encoding of the 7-bit ASCII code. The row corresponds to the lower four bits, the column to the upper three bits. The first two columns are special and control characters.

	0x0.	0x1.	0x2.	0x3.	0x4.	0x5.	0x6.	0x7.
0x.0:	NUL	DLE	SPACE	0	@	P	`	p
0x.1:	SOH	DC1	!	1	A	Q	a	q
0x.2:	STX	DC2	"	2	B	R	b	r
0x.3:	ETX	DC3	#	3	C	S	c	s
0x.4:	EOT	DC4	$	4	D	T	d	t
0x.5:	ENQ	NAK	%	5	E	U	e	u
0x.6:	ACK	SYN	&	6	F	V	f	v
0x.7:	BEL	ETB	'	7	G	W	g	w
0x.8:	BS	CAN	(	8	H	X	h	x
0x.9:	HT	EM	)	9	I	Y	i	y
0x.A:	LF	SUB	*	:	J	Z	j	z
0x.B:	VT	ESC	+	;	K	[	k	{
0x.C:	FF	FS	,	<	L	\	l	|
0x.D:	CR	GS	-	=	M	]	m	}
0x.E:	SO	RS	.	>	N	^	n	~
0x.F:	SI	US	/	?	O	_	o	DEL